Rapport annotation pipeline

This pipeline outlines the procedures for collecting and analysing rapport in dyadic interactions using video data. It will guide you through the data gathering, the setup process on both platforms Gorilla and Prolific and explain how they work together. The final section will be on data extraction and post-collection annotation processing.

Rapport annotation pipeline **1**
A) Dataset Structure, Alignment and Segmentation 2
Section 1: ELAN for media alignment 2
Section 2: Video segmentation 4
Section 3: Where the videos are stored? 5
Section 4: Randomisation of the videos and creating batches 5
B) How to set up a project on Gorilla? 5
Section 1: Gorilla 5
Section 2: Structure of the Task 6
Section 2: How to access full instructions? 6
Section 3: How does Gorilla task design work? 7
1) Practice Phase (Training Videos) 7
3) Main Evaluation Task 8
4) Final Questionnaire 8
Section 4: Further Instructions for setting up a project 8
C) How to set up a study on Prolific? 10
Login details: 10
Section 1: Data collection type 11
Section 2: Study details 11
Section 3: Data collection 11
Section 4: Recruit participants 11
Section 5: Cost 11
Section 5: Uploading funds to Prolific 12
D) Participant Evaluation 13
E) Post-collection annotation processing 13
Section 1: Data Extraction 13
Section 2: Data Aggregation 14
Section 3: Frequency calculation 14
Section 4: Outlier Removal 14
Section 5: Consolidated Score Calculation (weighted averaging) 15
Section 6: Utopy 16
Section 7: Concrete example of the entire rating process 17

A) Dataset Structure, Alignment and Segmentation

To start with the data structure, the adult video dataset is hierarchically structured around dyads of peers, each recorded over the course of three weeks (the children's data is recorded over the course of four weeks). In each week, dyads participated in a solo phase (not annotated) and two interaction phases: a collaboration phase, which involved a structured, task-oriented activity, and a free interaction phase, which captured spontaneous, natural conversation. Each phase lasted five minutes and was subsequently segmented into ten video clips of 30 seconds each during pre-processing.

Section 1: ELAN for media alignment

We used ELAN 6.7 to align the videos of the 2 participants (from five sources: 2 OBS video recordings, Webex screen recording, 2 microphone sound files) so that their temporal offsets matched. The goal was to synchronize all media with respect to a reference video, which in our case is the OBS recording of participant P1 (i.e., raw-data-adults-D01-M-W1-P1-video-obs.mp4). Once aligned, we used a Python script to segment the videos using the offset times extracted from ELAN.

For example, to crop the videos for participant D01 in Week 1, we followed the following steps:

1) First, download the following files from Box or the Inria server:

- raw-data-adults-D01-M-W1-P1-video-obs.mp4
- raw-data-adults-D01-M-W1-P2-video-obs.mp4
- raw-data-adults-D01-M-W1-P1-P2-video-webex.mp4
- raw-data-adults-D01-M-W1-P1-audio-microphone.wav
- raw-data-adults-D01-M-W1-P2-audio-microphone.wav

2) Launch ELAN 6.7 and import media:

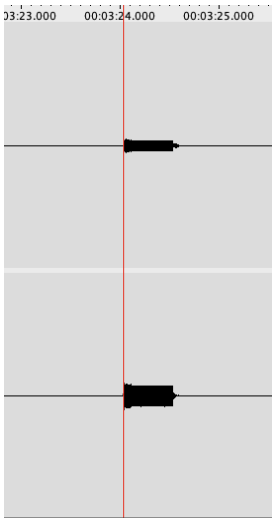
- Open ELAN.
- From the File menu, select New.
- Click Add Media File, then select the reference video (P1-video-obs.mp4) and confirm with OK.
- Enable Media Synchronization Mode under the Options tab.

3) In the player pane, assign the reference video to Player 1. Using the playback controls, manually locate the first frame where a white screen appears. This frame acts as our synchronization marker and is used to set the offset for all other media files (that is, to align all

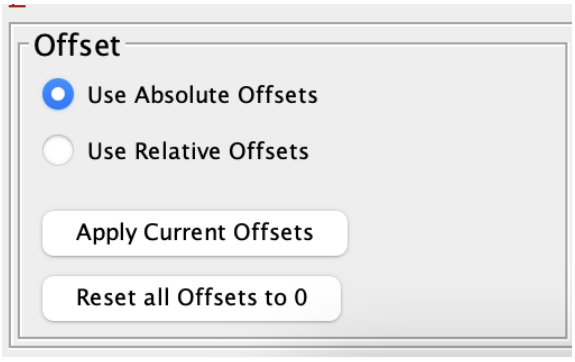
media). We chose the white screen because it is a consistent, easily recognizable visual cue across all recordings.

4) For each additional video (P2-video-obs.mp4, P1-P2-video-webex.mp4), assign it to a separate player (e.g., Player 2, Player 3), and manually adjust its offset so that the white screen aligns with that of the reference video.

5) For the audio files, use the waveform viewer to locate the first beep sound, which typically appears as a sharp, distinct waveform. This beep serves as the audio synchronization point and is aligned to the white screen marker in the video. Below is the visual waveform for the beep sound:



6) Once the alignment is complete, navigate to the “Offset” tab, select “Use Absolute Offsets”, and then click “Apply Current Offsets” to finalize the synchronization, as shown below:



7) Finally, it is time to export the alignment offsets into a .txt file to be used for video segmentation. Go to File → Export As → Tab-delimited Text. In the export options, make sure to include time values in milliseconds, then save the .txt file. This file is found under the folder

named data-processing-adults-Dyad[]-[gender]-week[] or example here is the link for D07-F-W3: https://cmu.box.com/s/ix5wg5tw3i28fkynj1o55nv6ospspr2d

Section 2: Video segmentation

For video segmentation, we use a Jupyter Notebook script named crop_video.ipynb. (link: https://cmu.box.com/s/947nj2an2j99mn8txmuzztwnk1mhwecj)

Continuing with our example, place the OBS video from participant P1 into your Jupyter environment. This video serves as the source for generating 30-second slices, provided the P1 OBS recording is available. Additionally, place the ELAN-exported .txt file into the same Python environment.

When you run the script, it creates a folder structure as follows:

```
data_analysis/
    video/
        by_phase/
        from_start/
        videos_30s/
            collaboration/
            free_discussion/
```

Within the videos_30s folder, the collaboration and free_discussion subfolders each contain 10 video clips, named using the following format:

Dyad[dyad number]_w[week number]_[phase]_[clip_number].mp4

- Dyad[dyad number] – the dyad identifier
- w[week] – the week/session number (1, 2, or 3)
- [phase] – either collab (collaboration) or free_discussion
- [clip_number] – from 0 to 9, for a total of 10 clips per phase

Each week yields 20 clips per dyad (10 per phase). Since there are 3 sessions per adult dyad (4 sessions for children), running this script across all sessions produces 60 clips per dyad in total.

Section 3: Where the videos are stored?

The data_analysis folder is renamed for each dyad and stored under data-processing folder in Box https://cmu.box.com/s/vtegkdlgytxc2ahwle86jgvykite5tgt → data-processing-adults → processed-data-adults-Dyad[dyad number]-[gender]-w[week]

Each clip is labeled using the format dyad[N]_w[week]_[phase]_[clip number] as explained just above. For example, dyad1_w1_collab_0 refers to the first collaboration clip of week one for dyad 1. This naming allows for systematic tracking of the clips across dyads, weeks, and phases.

Section 4: Randomisation of the videos and creating batches

For the rapport annotation task, the 60 "30-second clips" generated per adult dyad are randomly assigned into batches of five clips, randomised across dyads and weeks. These batches are then used in experiments hosted on Gorilla and distributed to participants via Prolific.

You can access the preprocessed files under the folder preprocessed-data-adults-Dyad1toDyad5 through the following Box link: https://cmu.box.com/s/rjm59romd5qqfcw5fupiftcq5l7b2sdf

This folder contains:

- Batches of five 30-second clips, covering all clips from Dyad 1 to Dyad 5 (folder: preprocessed-data-adults-Dyad1toDyad5-videos, link: https://cmu.box.com/s/uubecq1ksgjvh2gw201rij0o7cpf5tda)

- A randomisation Python script used to generate the clip batches(file: random-list maker.ipynb, link: https://cmu.box.com/s/i61xlxcdc4atggqyfumopm2z1hiahy86

    - This script should be run to generate batches for Dyads 6 to 22

- video_groups.txt, the output of the script, which lists all generated batches (link: https://cmu.box.com/s/947nj2an2j99mn8txmuzztwnk1mhwecj)

    - This list determines which videos are assigned to each Gorilla experiment (explained in the next section)
    - You need to update this file after you generate new batches.

B) How to set up a project on Gorilla?

Section 1: Gorilla

Gorilla email we use is articulabo@gmail.com

Password: Rapport2025

Prolific requires a third-party platform to host the experiment and display videos to participants. Two available options are Gorilla and PsychoPy (Pavlovia). Among these, Gorilla offers a more user-friendly interface and better integration with Prolific.

We use Gorilla with an academic pricing plan, where each token costs €1.22. One token allows one participant to complete the experiment.

To purchase tokens on Gorilla:

1. Go to My Account.
2. Click on Buy more tokens.
3. Specify the number of tokens you wish to purchase.
4. Proceed with the payment.

The payment is made by the team assistant using an *Inria* purchase card (*carte d'achat*).

Gorilla projects consist of two main components: the **Task** and the **Experiment**. The task defines what participants will see and do, while the experiment controls how the task is delivered and how participants are recruited. In our case, participants will be recruited through Prolific.

Section 2: Structure of the Task

The task is divided into the following components:

1. Instructions
2. Practice (Training)
3. Main Task
4. Debrief + Questionnaire

Section 2: How to access full instructions?

All the instructions can be found in the file named “Gorilla Adult study instructions” accessible on Articulabo drive using the following link:
https://docs.google.com/document/d/1gb0vyTShn1RNjtPakDLC17uw5GquF8S6ioDpi-KbDYY/edit?usp=sharing

Section 3: How does Gorilla task design work?

Participants are first introduced to the concept of rapport, or connivence in French. They are given a definition of what rapport is and are shown training videos of low, neutral and high rapport.

Definition of Rapport:

> *Le terme "connivence" décrit un certain type d'interaction entre deux personnes. Ce type d'interaction est souvent caractérisé par des expressions telles que « nous avons vraiment accroché » ou « nous avons été sur la même longueur d'ondes ». Des termes comme "harmonieux", ou "en phase" décrivent des interactions ayant une connivence élevée.*

Participants are then given instructions about their role in the study:

> *Votre tâche est d'évaluer le niveau de connivence pour une série de clips vidéo, chacune montrant une interaction entre 2 personnes. Vous allez voir des clips d'interactions de binômes différents, et l'ordre des clips a été aléatoirisé. Donc chaque tâche que vous faites vous montrera une tranche qui n'a rien à voir avec les autres tranches de vidéo que vous annoterez.*
>
> *On vous demande d'évaluer le niveau de connivence entre les deux personnes dans chaque vidéo de 30 secondes sur une échelle de 1 à 7, où: 1 = absence totale de connivence et 7 = connivence maximale.*
>
> *Vous devez également évaluer votre confiance dans votre annotation sur une échelle de 1 à 7, où: 1 = aucune confiance et 7 = confiance totale*

1) Practice Phase (Training Videos)

Before the real annotations, participants are shown 3 training videos representing different rapport levels. These are example videos that we have picked from the 30s slices extracted from the Pilot adult data. (URL: https://cmu.box.com/s/9g7l7rmdk8dms7gxwle29vadns95qf9m) We make sure participants watch these three videos before they move on to the next pages, this is ensured by showing the next button with a time delay, explained in detail in the trigger section. All three videos are under the folder preprocessed-data-adults-Dyad1toDyad5 under Gorilla_training_videos with the link https://cmu.box.com/s/abl0pz3ebk7i67b2bxk2150d47iql8hg

- One low rapport video

URL: https://cmu.box.com/s/77xtnzjhmhu9vn15qb0ssanw91jviuut

- One neutral rapport video

URL: https://cmu.box.com/s/cgqklyrpfij2czou04gyrtrpnnqrhb64

- One high rapport video

URL: https://cmu.box.com/s/pytidebwrlym94kwothuik1iagw85i8y

3) Main Evaluation Task

After training, participants will evaluate 5 real video clips, each 30 seconds long, using the same rapport and confidence rating scales.

4) Final Questionnaire

At the end of the task, participants must complete a brief open-ended questionnaire. They are asked to explain what factors influenced their evaluations for low, neutral, and high rapport clips.

Section 4: Further Instructions for setting up a project

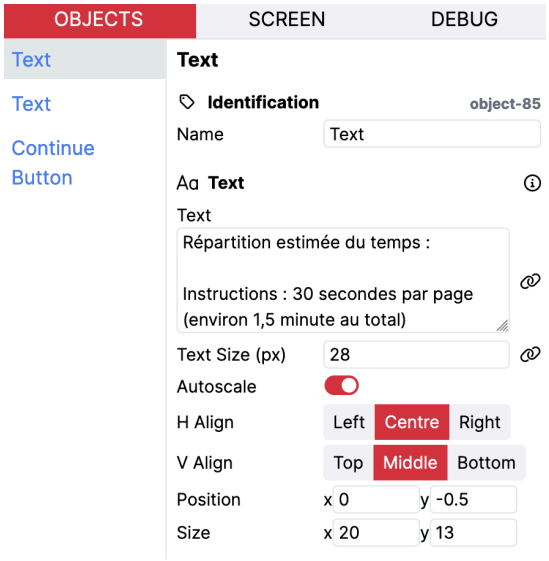
1. Create a New Project
    - Click on "Create New Project." URL: https://app.gorilla.sc/admin/projects
    - Rename the project to “Group [Experiment Number]” (e.g., *Group 1, Group 2*, etc.). Here the experiment or group numbers are named according to the video-groups.txt file under the folder "preprocessed-data-adults-Dyad1toDyad5”accessible with this link on box: https://cmu.box.com/s/947nj2an2j99mn8txmuzztwnk1mhwecj
2. Create a New Task
    - Inside the project, click on "Create New" → select "Task Builder."
    - Choose "Clone Existing."
    - Select the task from Group 1. Group 1 serves as the ground truth example, and accessible via https://app.gorilla.sc/admin/project/172179
    - Click on “Rapport.”
    - Click “Create.”

The cloned task contains four displays. The first display is titled "Instructions" and consists of four sequential instruction screens.

Each screen presents information to the participant. A “Continue” button appears in the bottom-right corner after a 10-second delay, allowing participants to proceed. This delay is time-triggered, ensuring participants have time to read before continuing.

The instruction content is added as "Text Objects." To modify or add text:

- Click “Edit” on the task.
- Navigate to the screen you want to change.
- Click “Add Object” → select “Text.”
- A window will appear on the right side of the page.
- Enter your content in Markdown format in the provided text box.

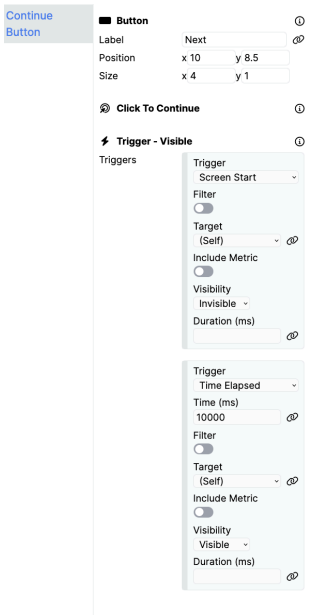


To allow participants to progress through the instruction screens, you’ll need to add a Continue Button Object that appears 10 seconds after the screen loads.

- In the Task Builder, go to the instruction screen where you want to add the button.
- Click “Add Object” → select “Continue Button.”
- A user-friendly settings panel will appear on the right side of the screen.

If you would like to add more triggers like this, in the Task Builder go to the Trigger Settings, you’ll need to set up when the button becomes visible:

- Trigger – Screen Start:
    - Metric: invisible
    - This ensures the button is not visible when the screen first loads.
- Trigger – Time Elapsed:
    - Metric: 10000 ms (10 seconds)
    - Action: visible
    - This ensures the button appears after 10 seconds, allowing the participant to proceed.



Section 5: Extracting the data log from Gorilla

To access the data log from Gorilla, go to the “Experiment” section, then to the “Data”tab. Click on, "Download Experiment Data”. Select “unblinded” data, this will allow you to save the data log with Public IDs of the participants. If you do not select this option it gets saved under a private ID, which makes it difficult to analyse after. Downloading data will give you raw experiment data, the which should be saved as GroupX.csv.

C) How to set up a study on Prolific?

Section 1: Data collection type

- Data collection type: Select “External study link”

Section 2: Study details

- Study name: Name it *“étude de rapport [experiment number]”*
- Internal study name: Leave blank
- Study description:
  “Dans cette étude, on vous demandera d'évaluer la connivence entre deux personnes.”
- Data collection type: Select “Annotation”
- Device requirements: Select computer
- Labels: None

Section 3: Data collection

- What’s the URL of the study? → Put in the URL you get from Gorilla
- Recording Prolific IDs → leave it on default, that is “URL parameters”
- Completion paths → leave it on default, that is “manually review"
    - Here, copy the URL that appears underneath “Redirect URL”. This will be used on Gorilla.

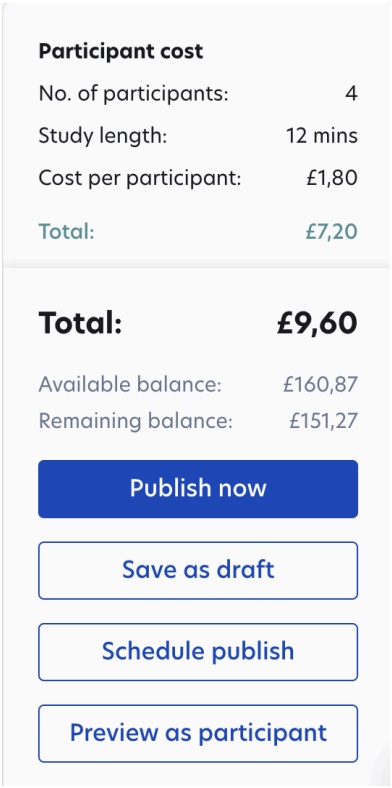
Section 4: Recruit participants

a) Source: select “Find new participants on Prolific”
b) Participants: enter the number “4”
c) Credentials: select no
d) Screening: select “use a saved screener set”. Here select the screener that is created under the name “Native French speakers living in France metropolitaine”. It will print the preselected country of residence as France and first language as French.
e) Submissions: Select “once”

Section 5: Cost

In France, we need to pay the equivalent of a SMIC corresponding to the time it takes for participants to complete our task. Our task takes 12 minutes and we pay £1.80 per participant.

Total cost of the experiment looks as the following:



When we click on the publish now, we run one experiment where the same 5 videos are shown to 4 different participants, i.e annotators. This experiment is linked to a Gorilla project with the corresponding name (For example if the Gorilla project is Group 28, then the Prolific study name is étude de rapport 28). As it is seen on the image above, in addition to the total costs per participant, the platform charges an additional £2,40 per experiment.

Section 5: Uploading funds to Prolific

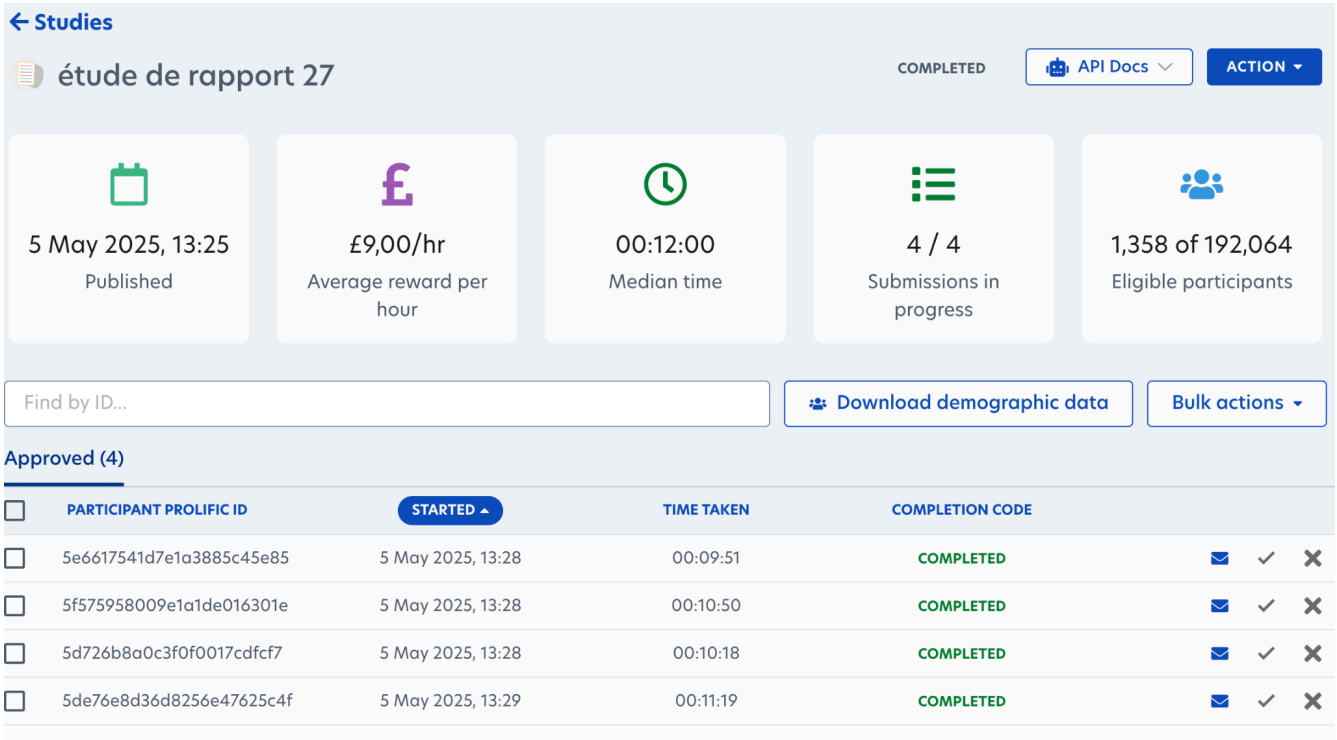
To upload funds to your Prolific account:

1. Log in to your Prolific account.
2. Click on "Finances" in the top navigation bar.
3. Under the "Add Funds" section, enter the desired amount you wish to upload.
4. Choose the appropriate payment method. For institutional accounts, this is typically done via invoice or payment card (*carte d'achat*).
5. Follow the instructions provided by Prolific to complete the transaction. **Inria can not pay for Prolific with a carte'dachat!!!**

D) Participant Evaluation

Section 1: Approving Participants

After the study is complete, review participant submissions on the Submissions tab in your Prolific dashboard.



You'll see if the participant has completed the study on the column that says "completion code". Here you also verify the time it has taken for a given participant to finish the study. As you can see in the above example, all participants finished the study around 10 minutes, and no one finished it under 9 minutes. Approve if the participant followed instructions and completed the task, using the tick on the right side of their respective row. Once you accept them, it means that you are approving Prolific to pay them.

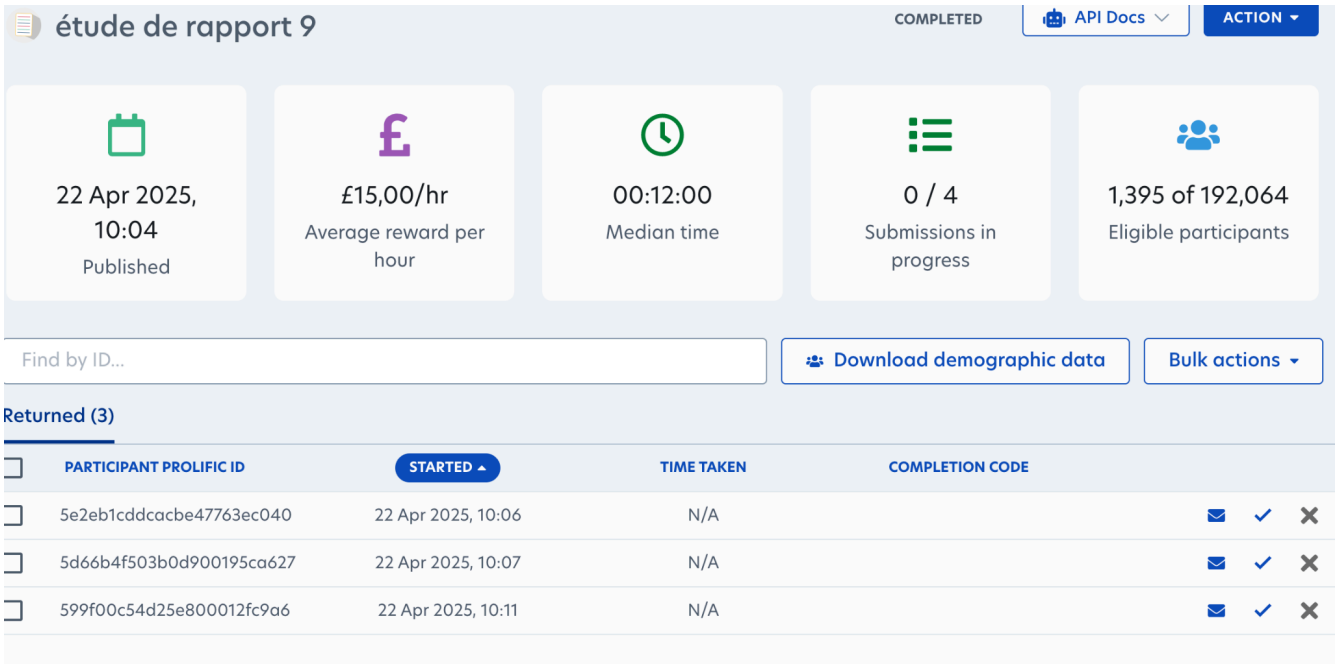
Check the time they have taken on the submission screen, normally everyone should be above 10 minutes as this is ensured in the experiment design with triggers. Check the quality of their answers to the final questionnaire, their answers to the final questionnaire can be found on the data log that is extracted from Gorilla.

Section 2: Returning Submissions

In some cases, you notice that the time taken seems “N/A”and there is no completion code associated with their participation. In such cases, click on the reject button. Prolific will then ask for a reason for your rejection and here you should select “Didn’t finish the study”.

Prolific automatically sends these participants the following message:

> You have been asked to return your submission for "connivence 10" for the following reason(s):
>
> - Didn't finish the study
>
> Please return the submission
> https://app.prolific.com/submissions/68109a36d6c46cee865563c5



Sometimes this may be due to an error from the experimenter side (putting in the wrong Gorilla link, or not airing the experiment on Gorilla. When you get such an error, go and check the Experiment tab of the Gorilla experiment associated with this study group.

Section 3: Excluding participant data- minimum time spent on the task

To assess the quality of participant engagement, we are interested in measuring how much time participants spend on a given slice after arriving on the page (i.e., after clicking “Next” from the previous screen). This helps us identify and potentially exclude participants who may have stayed on the screen but became distracted in other tasks before responding—behavior that indicates low attentiveness and could compromise data quality.

In the Gorilla data log file named GroupX.csv, there is a column called "Onset time", which provides the timestamp (in milliseconds) for when participants clicked on a response type. Each task slice contains **three responses**: a **"continue" response** (i.e., the "Next" button used to proceed to the following screen), and two additional responses categorized under **"response"**—specifically, the rapport rating and the confidence rating.

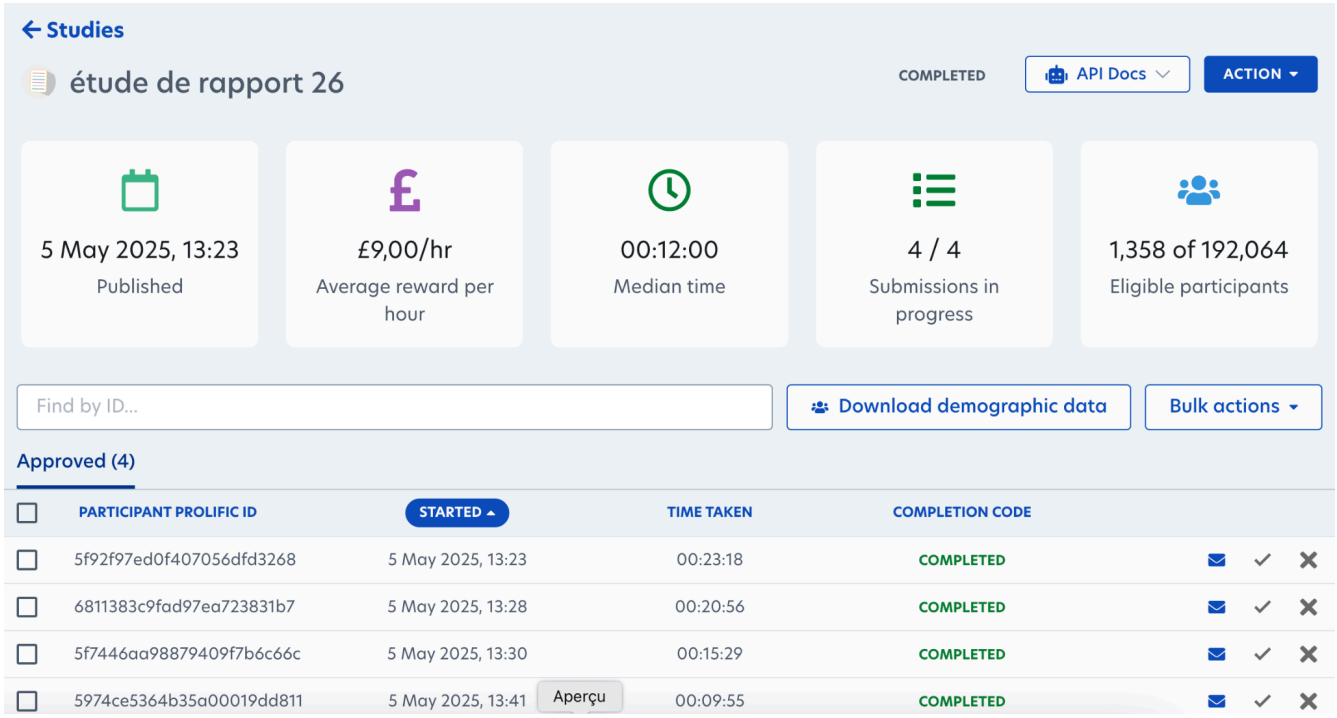
We do not have timestamp data for when participants clicked the play button on the video, but we do have the Onset time for the "continue" response and "response" responses. In the GroupX.csv (demonstrated below), these correspond to rows where the "Display" column is labeled "task". For example, rows 9- 23 should be checked to assess the quality of the first annotator’s work. In this given example, the participant responded around 35 seconds (34582 ms for the first slice (row 10), 35900 ms for the second slice (row 13), 35599 ms for the third slice (row 16) etc) after seeing the screen. Given our slices are 30 seconds to watch, this delay is short, however is still in the right time frame. If the response is given less than 30 seconds, this would indicate the participant didn't watch the videos fully and hence the rows associated with this annotator should be discarded.

On the other hand, if the annotator did spend too much time on a given screen (I don't have such an incidence for an example but it can happen). In case this happens for over 2.5 minutes, this annotator should also be discarded.

	Screen Counter	Response Type	Response	Onset Time
5	4	continue		65698.89999999997
6	1	continue		52899.40000000002
7	2	continue		38149.39999999927
8	3	continue		700801.8999999992
9	1	continue		51565.70000000228
10	1	response	5	34582.69999999995
11	1	response	7	36933.40000000014
12	1	continue		61048.60000000219
13	1	response	7	35900.09999999986
14	1	response	7	36683.5
15	1	continue		69014.9999999993
16	1	response	7	35599.90000000014
17	1	response	7	36617
18	1	continue		62083.59999999986
19	1	response	7	35516.39999999991
20	1	response	7	36533.5
21	1	continue		53148.800000000745
22	1	response	6	37133.5
23	1	response	7	38099.39999999991

Section 4: Excluding participant data- maximum time spent on the task

In this section, I present an example where the participant spent an unexpected amount of time 23 minutes on the experiment that is supposed to take 12 minutes on average. We first identify this abnormality on Prolific where the annotator ID 5f92f97ed0f407056dfd3268 took 23 :18 to complete the study:



In such a scenario, we go to the respective Gorilla data log (Group26.csv) and find the rows that belong to the annotator ID 5f92f97ed0f407056dfd3268. Extract the information from the "Onset time" column to see where in the design they exactly spent the abnormally long time. In this example, we find out that the participant spent 836159 ms (so 14 minutes):

	D	Screen Counter	Response Type	Response	Onset Time
1			info	BEGIN	
2	r2	1	continue		16026.199999999999
3	nl	2	continue		836159.7199999999

E) Post-collection annotation processing

After data collection via Gorilla, we follow several steps to extract, organize, and analyse the annotations:

Section 1: Data Extraction

Gorilla exports a .csv file containing metadata (e.g., task start time, video URL, reaction times). From this export, we extract the following key columns into a separate .csv file:

- Response (rapport score and confidence)
- Participant ID
- Video URL (to link the response to the correct clip)

Here is the link to access the files located on the Box:
https://cmu.box.com/s/e3jzf93wu03o5ixvz4d5qezy344yj0dy

Section 2: Data Aggregation

We combine responses across all annotators for a given dyad, aggregate data across multiple collection weeks, and then merge these into a single data frame.

Each data frame is composed of one dyad and contains :

- The dyad number
- The week of the experiment (week 1, week 2, week 3)
- The phase (collaboration or free_discussion)
- The slice (0 to 9)
- The annotators' ID number with their scores associated to this slice
- The URL of the slice
- The general mean
- The utopy (that we will describe further down)
- Krippendorff's alpha for this dyad
- The rapport score for a slice with weighted majority

Here's an exemple of what it should look like :

Dyade	Week	Phase	Slice	A1234	A5678	B	R	I	G	URL	mean_everys	Utopy	Krippendorf	Score final
D20	w1	collaboration	0	4	2	6	5	6	5	https://rappor	4,66666667			

Section 3: Frequency calculation

For each annotator, we compute the number of times each score (from 1 to 7) was used across all their annotations

For an annotator called ID1234 we would have something like this :

Annotateur	score 1	score 2	score 3	score 4	score 5	score 6	score 7
ID1234	13	31	22	24	14	9	7

Section 4: Outlier Removal

We implement an outlier removal procedure. We calculate the mean of all four ratings, then identify and discard the single rating that deviates most from this mean (i.e., has the largest absolute difference).

Exemple : [3, 4, 7, 4]

1. Compute the mean: (3 + 4 + 7 + 4) / 4 = 4.5
2. Compute absolute deviations from the mean: [1.5, 0.5, 2.5, 0.5]
3. Discard the score with maximum deviation (here, 7)

In case of a tie (e.g., the scores are: 1, 1, 6, 7), we resolve it by discarding the score from the annotator who has the highest overall frequency of the tied score.

For instance, if Annotators 1 and 2 both gave a score of 1, but Annotator 2 tends to overuse this score more frequently across their annotations, then Annotator 2's rating will be discarded based on this bias. In case of a tie in weights, we randomly discard one of the scores that make up the tie.

We then apply the weighted majority rule to the remaining three ratings to determine the final consolidated score.

Section 5: Consolidated Score Calculation (weighted averaging)

To derive consolidated rapport scores from the annotations, we use a weighted averaging approach that accounts for annotator reliability differences. First, we calculate inter-rater reliability using Krippendorff's alpha for the overall dataset level and for individual video segments to assess data quality and identify segments with poor annotator agreement. This is to see whether some slices should be excluded or interpreted more carefully. Krippendorff can be calculated through a script . Python supports a krippendorff package. Link to the script:https://drive.google.com/file/d/1mw66A3_Vp85xF4IeE4QG8mvSXqIEIHek/view?usp=drive_link

In parallel, we calculate a consolidated score using the weighted majority rule with reverse bias correction. That is, each annotator's rating is assigned a weight, defined as the inverse of the number of times the annotator used that score in all his annotations, 1/Freq_i(k). We calculate the total weighted score for each possible rating category by summing the weights of all annotators who selected that category by hand, then assigning the final rapport score k* as the category with the highest total weighted score. This could be achieved this by hand, by adding each annotator's weight for every score but we automated the process with python. Here is the link to the script: https://drive.google.com/file/d/15S0nkA2TcD7dob5r9qaGJXvmGkJ4pNsl/view?usp=drive_link

For an Annotator that used the “score 1” 13 times, their weight would be 1/13

For the same annotator called ID1234 we saw before, we would have those weights :

Annotateur	score 1	score 2	score 3	score 4	score 5	score 6	score 7
ID1234	0,07692308	0,03225806	0,04545455	0,04166667	0,07142857	0,11111111	0,14285714

For the first slice we had a score of 4 for annotator ID1234, so their weighted score here would be 1/24, or, 0,04166667.

We add up the weights associated with each score given by each annotator in a slice. The score with the highest total weight is selected as the final score. This approach helps reduce systematic bias from overused categories and produces a single, consolidated rapport rating per 30-second video segment.

Section 6: Utopy

Utopy is a metric used to quantify whether the rapport in a dyad tends to improve or deteriorate over time during an interaction. It is based on transitions between rapport levels observed over short time segments. The idea is not just to measure how often the rapport changes, but also how big those changes are. For example, moving from a rapport score of 2 to 4 is considered a stronger improvement than moving from 2 to 3.

To calculate the utopy, we first build a transition matrix. This is a square matrix where each cell (i, j) contains the probability that the rapport goes from level i to level j from one segment to the next.

The upper part of the matrix (where j > i) represents positive transitions (rapport improvements), and the lower part (where j < i) represents negative transitions (rapport declines). The diagonal is ignored since it indicates no change (1,1 ; 2,2 ; 3,3 etc).

Each transition is weighted by how large the change is (i.e., the absolute difference between j and i). For example, a transition from level 2 to 4 is weighted by 2, while a transition from level 3 to 2 is weighted by 1.

We then sum all the weighted values in the upper part of the matrix (Up) and all the weighted values in the lower part (Low). The final utopy score is calculated as:

$$Rpt_Utopy = (2 * (Up - Low)) / (n * (n - 1))$$

where n is the number of different rapport levels used during the session. This final score ranges from -1 to 1. A score close to 1 means rapport tends to improve across segments, a score close to -1 means rapport tends to deteriorate, a score near 0 means there is a balance between increases and decreases.

To avoid building the matrix by hand, and calculating every transition, which can be quite long the utopy can be calculated thanks to quick python script with the numpy package.
https://drive.google.com/file/d/1IDPZ-EUzzkZv18AC2r8bO88YZ9HNsODg/view?usp=drive_link

Section 7: Concrete example of the entire rating process

In this example, we deal with a single clip: dyad1_w1_collab_0. This clip has been annotated by four annotators: A1, A5, A9 and A3. The scores they awarded are as follows: A1 gave a 4, A5 gave a 4, A9 gave a 3, and A3 gave a 7.

For each annotator, we start by calculating the frequency of use of each score between 0 and 7, over the whole of their annotations (for example, over the 60 clips of a complete dyad). It is possible, and preferable, for one annotator to annotate an entire dyad; otherwise, ensure that each annotator annotates the same number of videos to maintain balance.
Let's assume that A1 used “score 4” 24 times, A5 used it 10 times, A9 gave “score 3” 13 times, and A3 used “score 7” 11 times.

Next, we identify the outlier to be removed. First, we calculate the average of the four scores: (4 + 4 + 3 + 7) / 4 = 4.5. We then calculate the absolute deviations from this average: the deviations are 0.5 for the two scores of 4, 1.5 for “score 3”, and 2.5 for “score 7”. The score that deviates furthest from the average is 7, so this is the one that is removed. Annotator A3 is therefore discarded for this clip.

This leaves three scores to process: 4 (from A1), 4 (from A5) and 3 (from A9). We now apply the weighted majority rule with correction for usage bias. To do this, we assign a weight to each annotation based on the frequency of use of the score concerned by the annotator. The weight is defined as the inverse of the number of times the annotator used that score in all his annotations. A1 used “score 4” 24 times, so its weight is 1 divided by 24, i.e. approximately 0.0417. A5 used this score 10 times, so its weight is 0.1. A9 used “score 3” thirteen times, so its weight is around 0.0769.

We now add up the weights associated with each score: “score 4” has a total weight of 0.0417 + 0.1 = 0.1417, while “score 3” has a total weight of 0.0769. The score with the highest total weight is selected as the final score. In this case, “score 4” is selected as the consolidated score for this clip.

This process is repeated for each clip of each dyad, and all the data is centralized in a data frame file, where each line corresponds to a 30-second clip with all the information mentioned.

In parallel, we also calculate Krippendorff's alpha for the data frame, and each specific clips based on the four initial annotations (after exclusion of the outlier), enabling us to assess inter-annotator consistency. This measure is used to identify clips that may be unreliable.

Finally, we can compute the utopy value. This can be done for the entire interaction of the dyad, using all 20 segments, or separately for each phase, using 10 segments per phase, to obtain more detailed insights.

Suppose the rapport scores for the 10 segments are as follows:
3, 4, 5, 5, 6, 4, 3, 4, 6, 7

Now, let's look at the transitions between each consecutive segment:

From segment 1 to 2: 3 → 4 (positive change of 1)
From segment 2 to 3: 4 → 5 (positive change of 1)
From segment 3 to 4: 5 → 5 (no change)
From segment 4 to 5: 5 → 6 (positive change of 1)
From segment 5 to 6: 6 → 4 (negative change of 2)
From segment 6 to 7: 4 → 3 (negative change of 1)
From segment 7 to 8: 3 → 4 (positive change of 1)
From segment 8 to 9: 4 → 6 (positive change of 2)
From segment 9 to 10: 6 → 7 (positive change of 1)

Next, we build a 7x7 transition matrix representing the count of transitions from each score i to score j. Here, the scores range from 1 to 7, but only scores 3, 4, 5, 6, and 7 appear in the data.

Counting the transitions:

Positive transitions (where j > i):
3 → 4: occurs twice (segments 1→2 and 7→8), each with change of 1 → weighted sum = 2 × 1 = 2
4 → 5: once, change of 1 → weighted sum = 1
5 → 6: once, change of 1 → weighted sum = 1

4 → 6: once, change of 2 → weighted sum = 2
6 → 7: once, change of 1 → weighted sum = 1

Sum of weighted positive transitions (Up) = 2 + 1 + 1 + 2 + 1 = 7

Negative transitions (where j < i):
6 → 4: once, change of 2 → weighted sum = 2
4 → 3: once, change of 1 → weighted sum = 1

Sum of weighted negative transitions (Low) = 2 + 1 = 3

Number of unique rapport levels used is 5 (scores 3, 4, 5, 6, 7), so n = 5

Now, calculate the utopy score:

Rpt_Utopy = 2 × (Up − Low) / (n × (n − 1))
Rpt_Utopy = 2 × (7 − 3) / (5 × 4)
Rpt_Utopy = 2 × 4 / 20 = 8 / 20 = 0.4

Interpretation: A utopy score of 0.4 indicates a moderate tendency for rapport to improve over the course of the video, despite some decreases.